

CLIMB: CURRICULUM LEARNING FOR MULTI-AGENT BEHAVIOR TREE GENERATION WITH SMALL LANGUAGE MODELS

Yichen Wang¹, Zhongxuan Cai^{*1}, YINUO Liu¹, Yuhao Wang¹, Tianjian Jiang¹, Yuanxi Peng¹

¹College of Computer Science and Technology,
National University of Defense Technology, Changsha, China

ABSTRACT

Behavior trees have become a popular approach for modeling robot behaviors through recursive tick signals, but their automated construction remains challenging. Recent work explores using large language models (LLMs) to generate behavior trees from natural-language task descriptions. However, in multi-agent coordination scenarios, LLMs face critical challenges: they struggle to learn coordination primitives, are too large for practical deployment, and lack spatial cognition, causing task failures. Addressing these challenges, we present **CLIMB** (Curriculum Learning for Multi-agent Behavior Tree Generation with Small Language Models): at the supervision stage, we leverage symbolic planning algorithms to construct high-quality training data encoding correct multi-agent cooperation priors; we further optimize both training and inference stages—at the training stage, we design a curriculum learning strategy that identifies coordination complexity from both difficulty and utility perspectives to progressively improve learning effectiveness; at the inference stage, we provide the model with spatial cognition to prevent generating spatially unreachable behaviors. We validate our approach through both numerical simulation and physics-based 3D simulation in ROS 2/Gazebo, demonstrating that our method significantly improves execution success rate compared to standard supervised fine-tuning, and our fine-tuned 1B small language model outperforms current mainstream LLMs while reducing token cost. Our code is available at <https://yichenwang2002.github.io/CLIMB/>.

Index Terms— Behavior Trees, Multi-Agent Coordination, Curriculum Learning, Constrained Decoding, Execution Evaluation

1. INTRODUCTION

Behavior trees (BTs) have become the popular way to organize complex robot behavior in modern robotics [1, 2]. Compared with finite-state machines, their modular, hierarchical structure unifies reactive control, sequential tasks, and fault recovery, driving adoption in commercial platforms, game AI, and multi-robot systems [3].

Automating BT construction has traditionally relied on symbolic planning [4, 5], evolutionary algorithms [6, 7], or reinforcement learning [8], each of which needs a hand-crafted domain model. Large language models (LLMs) promise a more general route [9, 10]: generating BTs directly from natural-language task descriptions using pretrained world knowledge.

However, as we demonstrate in this paper, LLMs fail at multi-agent behavior tree generation. The failures stem from two key issues. First, LLMs struggle to learn multi-agent coordination primitives: a team tree must align per-agent subtrees under `Parallel`,

bind joint-action nodes to explicit agents and objects, and recover via fallbacks—global topology constraints that next-token prediction rarely satisfies, so trees may parse yet deadlock or drop hand-offs. Second, LLMs lack spatial cognition of the target platform: actions must reference entities that exist and are reachable, yet models readily emit unreachable moves or nonexistent references. Neither few-shot prompting nor scaling up models resolves these issues, and invoking large LLMs at execution time stalls the robot.

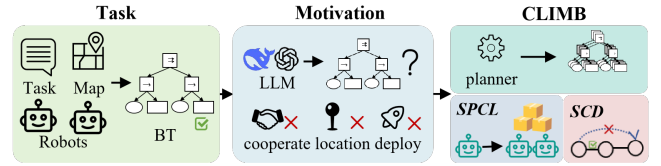


Fig. 1. CLIMB enables a small language model to generate multi-agent behavior trees.

We propose **CLIMB** (Curriculum Learning for Multi-agent Behavior Tree Generation with Small Language Models) (Fig. 1), which grounds supervision, training, and inference in symbolic planning: executor-validated data supplies correct cooperation priors, a curriculum orders examples by coordination complexity, and constrained decoding grounds generation in the platform map. A 1B model trained and constrained this way surpasses 5-shot LLMs at one-tenth the inference cost, achieving the highest success rate in both numerical simulation and physics-engine-driven ROS 2/Gazebo simulation. The main contributions are:

- **A planner-grounded multi-agent behavior tree dataset.** We construct executor-validated mission-tree pairs through symbolic planning and behavior tree expansion, covering complex coordination tasks that LLM-generated data cannot reliably solve.
- **Structure-Paced Curriculum Learning (SPCL).** We design a curriculum that orders training examples by difficulty and utility, improving the model’s learning of multi-agent coordination primitives.
- **Symbolic Constrained Decoding (SCD).** We enhance the model’s spatial cognition at inference.
- **Comprehensive validation.** We validate CLIMB along two dimensions: large-scale numerical simulation that measures execution success rate and token cost on held-out coordination tasks, and physics-engine-driven ROS 2/Gazebo simulation with physical TurtleBot3 robots that measures real task completion, ensuring that the reported gains reflect both symbolic correctness and deployable behavior.

^{*}The corresponding author. Contact Email: caizhongxuan@nudt.edu.cn

2. RELATED WORK

LLM behavior-tree generation. BTGenBot [9] fine-tunes lightweight LLMs on language–BT pairs, and BTGenBot-2 [11] improves efficiency. Other works prompt general LLMs for direct generation [10], social-robot adaptation [12], tree expansion [13], sketch-guided synthesis [14], and multi-robot dialogue [15], but outputs may parse yet fail during execution, and knowledge-graph grounding [16] does not validate execution. MRBTP [17] uses an LLM only for action pre-planning around a symbolic planner. In contrast, we validate every multi-agent training pair with a planner and executor, and train a small model with curriculum learning and constrained decoding.

LLM-based multi-agent collaboration. Existing systems coordinate embodied agents via messages, deliberation, or centralized planning [18, 19, 20], keeping communication or LLM/planner calls on the critical path, while multi-agent RL learns teammate policies [21]. Behavior trees instead execute through a periodic tick from the root [2]. Our single generated tree encodes cooperation in its topology and joint-action nodes, so execution needs only the recursive tick, with no LLM inference in the loop.

Curriculum learning and data selection. Classical curricula order examples from easy to hard [22]. LLM-era methods refine this at different granularity: skill-level mixing [23], instruction selection by utility [24], and token-level scoring [25]. DUCL [26] shows difficulty alone is insufficient and jointly scores utility, motivating SPCL’s joint difficulty–utility design for multi-agent BT generation.

3. PRELIMINARIES AND PROBLEM FORMULATION

3.1. Preliminaries

A behavior tree is a rooted hierarchical policy for robot behavior [2]. Internal nodes define control flow:  Sequence ticks children left-to-right and succeeds only if all succeed,  Fallback succeeds if any child succeeds, and  Parallel ticks children concurrently. Leaves are either  Conditions, which test predicates, or  Actions, which execute robot commands and may return Running (Fig. 2(a)).

Execution proceeds through a periodic tick: each control period the root sends a tick, and every ticked node returns Running, Success, or Failure to its parent. For a team, one tree uses Parallel branches for per-agent subtrees, and joint-action and synchronization nodes carry explicit agent and object identifiers; cooperation is thus encoded in the tree topology and exercised by ticks alone (Fig. 2(b)).

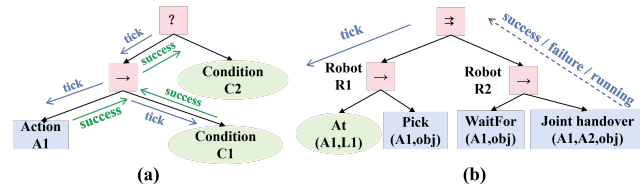


Fig. 2. Behavior-tree preliminaries. (a) BT structure and tick–status signal flow. (b) Two-robot relay with a joint action.

3.2. Problem Formulation

A mission is $x = (x_{\text{usr}}, x_{\text{sys}})$, where x_{usr} is a natural-language instruction and x_{sys} describes the platform, from which we extract a

Algorithm 1: Forward-Search Dataset Construction

Input: Domain $\mathcal{D}$; target size N .

Output: Validated dataset $\mathcal{S}$.

```

1  $\mathcal{S} \leftarrow \emptyset$ ;
2 while  $|\mathcal{S}| < N$  do
3    $\tau \leftarrow \text{SampleTask}(\mathcal{D})$ ;
4    $\pi \leftarrow \text{ForwardSearch}(\text{NominalDomain}(\tau))$ ;
5    $\tau' \leftarrow \text{InjectFaults}(\tau)$ ;
6   if  $\neg \text{ReplayReachable}(\pi, \text{FullDomain}(\tau'))$  then
7     continue;
8    $y \leftarrow \text{CompileBT}(\pi, \tau')$ ;
9   if  $\text{Executor}(y, \tau')$  then
10     $\mathcal{S} \leftarrow \mathcal{S} \cup \{(\text{Paraphrase}(\tau'), y)\}$ ;
11 return  $\mathcal{S}$ 

```

platform map $\mathcal{M} = (\mathcal{R}, \mathcal{I}, \mathcal{L}, \mathcal{E}, \mathcal{C})$ with sets of robots, objects, locations, connectivity edges, and charging stations. Our goal is to learn a small generator g_θ that maps $(x, \mathcal{M})$ to an executable behavior tree y coordinating the team through per-agent subtrees and joint-action nodes; the locations, robots, and objects referenced by y must exist in $\mathcal{M}$.

A symbolic executor $\Phi(y; x) \in \{0, 1\}$ returns 1 iff y is syntactically correct, executes without errors on $\mathcal{M}$, and reaches all mission goals. On a held-out test set $\mathcal{T}$, success rate is

$$\text{SR}(\theta) = \frac{1}{|\mathcal{T}|} \sum_{x \in \mathcal{T}} \Phi(g_\theta(x, \mathcal{M}); x).$$

Given training pairs $\mathcal{D}$ of missions and target behavior trees, we train g_θ with teacher forcing

$$\mathcal{L}(\theta) = -\mathbb{E}_{(x,y) \sim \mathcal{D}} \sum_t \log p_\theta(y_t \mid x, y_{<t}, \mathcal{M}),$$

where t indexes token positions.

4. METHOD

4.1. Dataset

Each sampled task τ (Algorithm 1, Fig. 3) specifies a platform description x_{sys} , an initial state, a goal, and a connectivity graph, from which we extract the platform map $\mathcal{M}$. A symbolic forward search on a nominal domain without recovery actions yields a plan π ; after fault injection, π is replayed on the full domain containing recovery actions and kept only if the goal remains reachable (recoverable perturbations, not plan-breaking failures), then compiled into a behavior tree y with parallel branches, joint-action rendezvous nodes, and fallback branches from the action vocabulary (hand-over retry, edge re-navigation, recharge). Only trees passing the symbolic executor Φ are retained and paired with a paraphrased instruction x_{usr} , giving training pairs (x, y) whose targets are executor-validated by construction.

4.2. Structure-Paced Curriculum Learning

In general, fine-tuning presents pairs in a fixed order. Structure-Paced Curriculum Learning (SPCL) performs curriculum learning from two perspectives: difficulty and utility.

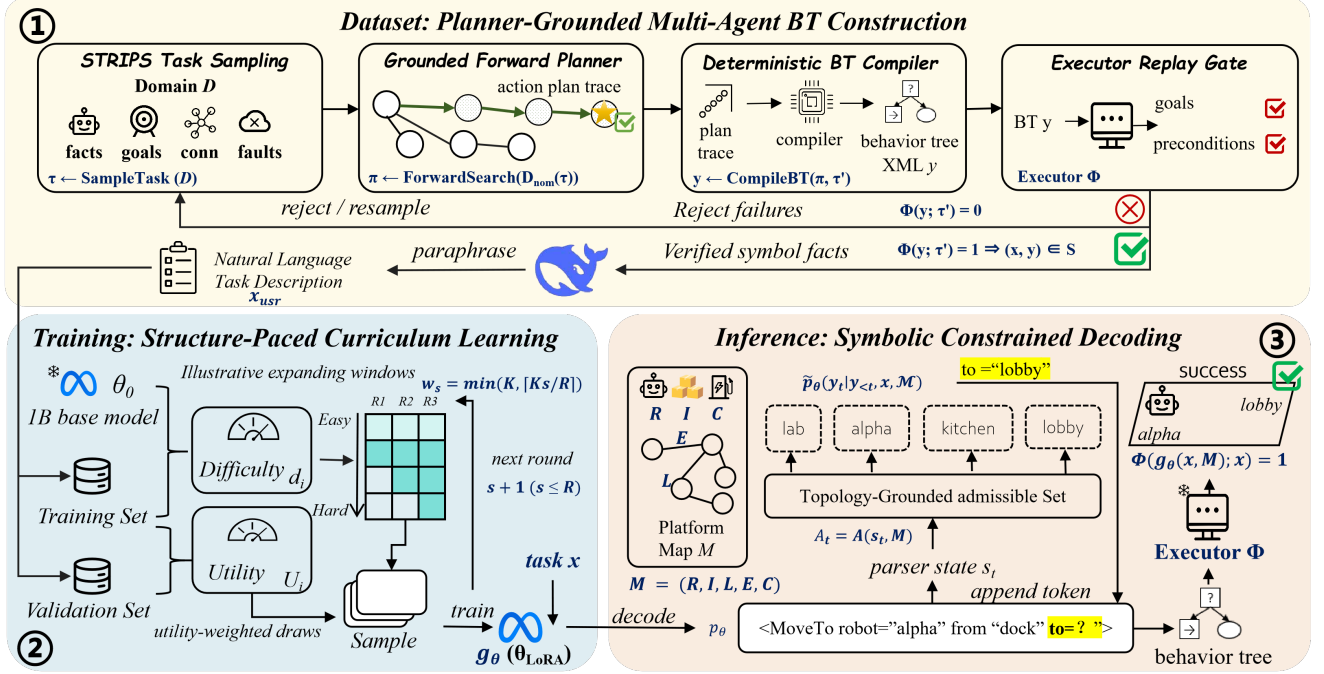


Fig. 3. Overview of CLIMB. ① **Supervision:** planner-grounded construction of executor-validated mission-tree pairs. ② **Training:** SPCL paces rounds with expanding windows ranked by difficulty and utility. ③ **Inference:** SCD restricts each decoding step to map-consistent location tokens, preventing spatially invalid outputs.

Difficulty. We measure difficulty from two views: a semantic view using the model’s negative log-likelihood and top-1 tag error, and a structural view using tree-level counts (number of robots, faults, synchronization nodes, joint-action nodes, fallback nodes, tree depth, and total nodes). Let Z be the feature matrix in one view, with row Z_i and entry Z_{ij} . We fuse its p columns as

$$\text{Fuse}(Z_i; Z) = \sum_{j=1}^p \alpha_j \rho(Z_{ij}), \quad \alpha_j = \frac{|v_j|}{\sum_{\ell=1}^p |v_\ell|}, \quad (1)$$

where p is the number of features in the view, ρ is mid-rank normalization to $[0, 1]$, and $v = (v_1, \dots, v_p)^\top$ is the leading eigenvector of the rank-normalized covariance. Absolute values fix the eigenvector sign, as all features are oriented so larger values harder; the weights favor the dominant shared-variance direction, with equal weights used only when v is degenerate (single feature or zero loading), not for uncorrelated features. We fuse within each view and across views to obtain $d_i \in [0, 1]$.

Utility. We bias sampling toward examples close to the validation set. For view $v \in \{\text{semantic (text), structural (tree)}\}$ and validation set $\mathcal{V}$, we compute a Gaussian kernel similarity to the k nearest validation neighbors,

$$u_i^{(v)} = \frac{1}{k} \sum_{j \in \mathcal{N}_k(i, \mathcal{V})} \exp\left(-\delta_{ij}^{(v)2} / 2\sigma_v^2\right), \quad (2)$$

where $\mathcal{N}_k(i, \mathcal{V})$ is the set of k nearest validation examples to training sample i , $\delta_{ij}^{(v)}$ is the embedding distance between i and neighbor j in view v , and σ_v is the median of all positive distances from training samples to their nearest validation neighbors in view v . The two

view scores are mid-rank normalized and fused into U_i using the same PCA-loading rule as for difficulty.

Schedule. We sort examples by d_i into K equal-mass buckets and, across stages $s = 1, \dots, R$, sample from an expanding window $w_s = \min(K, \lceil Ks/R \rceil)$, drawing with probability proportional to $U_i m_{s,b_i}$, where b_i is the bucket of example i and $m_{s,b}$ is the multiplier for bucket b at stage s . The multipliers are uniform in stage 1, $m_{s,b} \propto 1/\bar{U}_b$ in intermediate stages, and $m_{R,b} \propto \bar{D}_b/\bar{U}_b$ in the final stage, with $\bar{D}_b, \bar{U}_b$ the per-bucket means of d_i, U_i . The per-bucket target proportions are rounded to multiples of $1/K$ and lower-bounded by $1/K$.

4.3. Symbolic Constrained Decoding

At inference time, SCD grounds generation in the current platform map $\mathcal{M} = (\mathcal{R}, \mathcal{I}, \mathcal{L}, \mathcal{E}, \mathcal{C})$, preventing outputs that violate the target environment. Let s_t be the parser state after prefix $y_{<t}$, which knows the expected spatial field (move source, move target, or location argument). SCD constructs the admissible set $A_t = A(s_t, \mathcal{M})$ by keeping only location identifiers in $\mathcal{L} \cup \mathcal{C}$ and allowing a move $a \rightarrow b$ only when $(a, b) \in \mathcal{E}$. The constrained distribution is

$$\tilde{p}_\theta(y_t | y_{<t}, x, \mathcal{M}) = \text{softmax}(\ell_t - \infty \cdot \mathbf{1}[\cdot \notin A_t]),$$

where ℓ_t are the logits at step t . Tokens outside A_t receive zero probability, so locations lie in $\mathcal{L} \cup \mathcal{C}$ and moves in $\mathcal{E}$; SCD guarantees only this spatial consistency—grammar, identifiers, and action-schema validity are learned rather than enforced, and the executor flags residual violations as failure, so SR reflects end-to-end executability.

5. EXPERIMENT

5.1. Experimental setup

Tasks and scenarios. Given a natural-language mission and the platform map $\mathcal{M}$ for a team of 2–3 robots, the system must output one executable behavior tree. Training and validation use four seen domains (warehouse, hospital, search–rescue, office); testing uses two unseen domains (library, greenhouse) on a 480-task coordination test set—320 relay-transport and 160 joint-heavy-transport missions, 357 with injected recoverable faults. We further validate CLIMB on 38 coordination missions in ROS 2/Gazebo with 2–3 TurtleBot3 robots.

Verification and metrics. In numerical simulation, a generated behavior tree is counted as successful only if it is syntactically valid, parses, and ticks to *SUCCESS*. We report execution success rate and average token cost per task on the 480 held-out tasks. In the 3D robotic tasks, we measure task completion rate.

Baselines. All trained systems use Llama-3.2-1B-Instruct as the base model. BTGenBot-MA adapts the BTGenBot [9] supervised fine-tuning recipe to our format and serves as the training-side reference. We further compare against four prompted general-purpose LLMs (1/5-shot, $T=0$, no retry/repair/map mask; spatial masking is our SCD component) and validate CLIMB on a comparable-size model, DeepSeek-R1-Distill-Qwen-1.5B (Table 1). As a symbolic reference, MRBTP [17] runs its official planner without LLM subtree plugin on oracle nominal-task formalization; it receives no natural language or fault inputs, native fallbacks and joint actions are preserved, and outputs are evaluated via the shared symbolic executor without reference plans or target trees. All prompts, decoding settings, and training/curriculum hyperparameters are fixed across baselines and released with the code.

5.2. Results

Numerical simulation. Despite oracle symbolic input, MRBTP lags: it lacks recovery actions and stalls on tightly coupled three-robot tasks. CLIMB achieves the highest success rate, outperforming all four prompted LLMs under 5-shot and improving over BTGenBot-MA on both backbones at token costs well below prompted LLMs (Table 1).

Table 1. Comparison with baselines on multi-agent BT generation.

Model	SR (%)		Tokens	
	1-shot	5-shot	1-shot	5-shot
GPT-5.6-Luna	6.46	50.42	2,678	6,366
Kimi-K2.5	8.12	54.38	1,813	5,981
DeepSeek-V4-Flash	10.83	59.17	1,799	9,024
Qwen3.5-Plus	20.62	60.63	6,628	10,686
BTGenBot-MA[9]				
Llama3.2-1B-Instruct	58.96		1,102	
DeepSeek-R1-Distill-Qwen-1.5B	38.75		1,076	
MRBTP-Adapted [17]	42.08		–	
CLIMB (ours)				
Llama3.2-1B-Instruct	67.92		1,104	
DeepSeek-R1-Distill-Qwen-1.5B	62.92		1,092	

3D simulation. CLIMB completes the most missions and matches or exceeds all four 5-shot LLMs at substantially lower token cost; all

Table 2. ROS 2/Gazebo task completion.

System	Complete	SR (%)
GPT-5.6-Luna (5-shot)	19/38	50.0
Kimi-K2.5 (5-shot)	24/38	63.2
DeepSeek-V4-Flash (5-shot)	20/38	52.6
Qwen3.5-Plus (5-shot)	21/38	55.3
BTGenBot-MA[9]	19/38	50.0
CLIMB(ours)	27/38	71.1

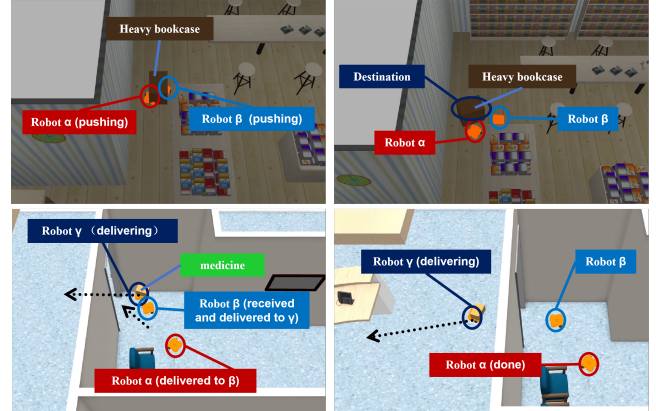


Fig. 4. ROS 2/Gazebo task completion under CLIMB.

executor-verified trees also complete in this 3D setting, confirming the numerical gains transfer to practical execution (Table 2, Fig. 4).

5.3. Ablation Study

Table 3 removes one component at a time. Replacing planner supervision with 5-shot LLM regeneration (4,160 targets at 69.3% yield, +49.6M generation tokens) causes the largest drop on both backbones; removing SPCL or SCD also degrades SR at unchanged token cost, and every component contributes more on the weaker 1.5B backbone.

Table 3. Component ablation. ([†]DeepSeek planner-free targets.)

Configuration	Llama-3.2-1B		R1-Distill-1.5B	
	SR (%)	Tokens	SR (%)	Tokens
w/o planner superv. [†]	48.96	1,090	51.3	1,081
w/o SPCL	63.75	1,107	50.6	1,094
w/o SCD	64.17	1,098	51.3	1,075
CLIMB (ours)	67.92	1,104	62.9	1,092

6. CONCLUSION

We presented CLIMB, which turns natural-language multi-agent missions into executable behavior trees via planner-grounded supervision, SPCL difficulty–utility curricula, and SCD spatial grounding. Numerical and ROS 2/Gazebo simulations show gains over prompted LLMs and supervised baselines at lower token cost. Future work extends to larger heterogeneous teams and real robots.

7. REFERENCES

- [1] Michele Colledanchise and Petter Ögren, *Behavior Trees in Robotics and AI: An Introduction*, CRC Press, 2018.
- [2] Matteo Iovino, Edwards Scukins, Jonathan Styrud, et al., “A survey of behavior trees in robotics and ai,” *Robotics and Autonomous Systems*, vol. 154, pp. 104096, 2022.
- [3] Petter Ögren, “Behavior trees in robot control systems,” *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 5, pp. 81–107, 2022.
- [4] Michele Colledanchise, Diogo Almeida, and Petter Ögren, “Towards blended reactive planning and acting using behavior trees,” *arXiv preprint arXiv:1611.00230*, 2016.
- [5] Zhongxuan Cai, Minglong Li, Wanrong Huang, and Wenjing Yang, “BT expansion: A sound and complete algorithm for behavior planning of intelligent robots with behavior trees,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2021, vol. 35.
- [6] Pei Zhang, Joe Collenette, Jakob Dahlberg Larsson, et al., “Learning behavior trees for autonomous agents with hybrid constraints evolution,” *Applied Sciences*, vol. 8, no. 7, pp. 1077, 2018.
- [7] Michele Colledanchise, Ramviyas Parasuraman, and Petter Ögren, “Learning of behavior trees for autonomous agents,” *IEEE Transactions on Games*, pp. 183–189, 2019.
- [8] Jonathan Styrud, Victor Karlsson, Alexander Lidström, et al., “Combining planning and learning of behavior trees for robotic assembly,” in *2022 International Conference on Robotics and Automation (ICRA)*. IEEE, 2022, pp. 11511–11517.
- [9] Riccardo Andrea Izzo, Gianluca Bardaro, and Matteo Matteucci, “Btgenbot: Behavior tree generation for robotic tasks with lightweight llms,” in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024, pp. 9684–9690.
- [10] Jiong Ao, Fan Wu, Yansong Wu, et al., “Llm-as-bt-planner: Leveraging llms for behavior tree generation in robot task planning,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 1233–1239.
- [11] Riccardo Andrea Izzo, Gianluca Bardaro, and Matteo Matteucci, “Btgenbot-2: Efficient behavior tree generation with small language models,” *arXiv preprint arXiv:2602.01870*, 2026.
- [12] Sergio Merino-Fidalgo, Celia Sánchez-Girón, Eduardo Zalamá, et al., “Behavior tree generation and adaptation for a social robot control with LLMs,” *Robotics and Autonomous Systems(RAS)*.
- [13] Jonathan Styrud, Matteo Iovino, Mikael Norrlöf, Petter Ögren, and Christian Smith, “Automatic behavior tree expansion with llms for robotic manipulation,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [14] Wenmeng Zhang, Zhenbang Chen, and Weijiang Hong, “Multi-modal sketch-based behavior tree synthesis,” *Proceedings of the ACM on Programming Languages*.
- [15] Artem Lykov, Maria Dronova, Naglov Nikolay, et al., “Llm-mars: Large language model for behavior tree generation and nlp-enhanced dialogue in multi-agent robot systems,” *arXiv preprint arXiv:2312.09348*, 2023.
- [16] Jianchao Yuan, Shuo Yang, Qi Zhang, et al., “Trustworthy robot behavior tree generation based on multi-source heterogeneous knowledge graph,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2025.
- [17] Yishuai Cai, Xinglin Chen, Zhongxuan Cai, et al., “Mrbtp: Efficient multi-robot behavior tree planning and collaboration,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2025, Oral presentation. arXiv:2502.18072.
- [18] Hongxin Zhang, Weihua Du, Jiaming Shan, et al., “Building cooperative embodied agents modularly with large language models,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [19] Zhao Mandi, Shreeya Jain, and Shuran Song, “Roco: Dialectic multi-robot collaboration with large language models,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 286–299.
- [20] Shyam Sundar Kannan, Vishnunandan L. N. Venkatesh, and Byung-Cheol Min, “Smart-llm: Smart multi-agent robot task planning using large language models,” in *IEEE/RSJ IROS*, 2024, pp. 12140–12147.
- [21] Yinuo Liu, Zhongxuan Cai, Tianjian Jiang, et al., “T-camel: Teammate-causal-aware multi-agent learning,” in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2026.
- [22] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, et al., “Curriculum learning,” in *International Conference on Machine Learning (ICML)*, 2009.
- [23] Mayee F. Chen, Nicholas Roberts, Kush Bhatia, et al., “Skill-it! a data-driven skills framework for understanding and training language models,” in *Advances in Neural Information Processing Systems(NeurIPS)*, 2023, pp. 36000–36040.
- [24] Ishika Agarwal, Krishnateja Killamsetty, Lucian Popa, et al., “Delift: Data efficient language model instruction fine-tuning,” in *International Conference on Learning Representations (ICLR)*, 2025, pp. 100488–100509.
- [25] Jing Pang, Nan Di, Zinan Zhu, et al., “Token cleaning: Fine-grained data selection for llm supervised fine-tuning,” in *International Conference on Machine Learning (ICML)*, 2025.
- [26] Zishang Jiang, Jinyi Han, Tingyun Li, et al., “Difficulty is not enough: Curriculum learning for llms fine-tuning must consider utility,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026.